

Relatório — Fase 0 (concluída) e Fase 1 (a seguir)

Projeto: LangNet · caso-teste ClinIA · **Data:** 2026-08-12/13 · **Executor:** Claude **Plano de referência:** PLANO-IMPLEMENTACAO-v3.md · **Especificação:** ESPEC-SPECIFICATION-ENGINEERING-v3.md

Este relatório documenta **o que foi feito e validado na Fase 0**, com as **saídas reais dos testes**, e **o que será feito na Fase 1**. Nota importante: **a Fase 0 não altera nenhuma tela** — é apenas ferramental de apoio. As telas mostradas abaixo são **prova de que o smoke-test exercitou a UI real**.

1. Resumo executivo

- **Fase 0 — CONCLUÍDA E VALIDADA** (commit `e7b595f`). Entregou um **harness de regeneração + smoke-test** versionado (`tools/regen/`), tornando o ciclo *editar-gerador* → *regenerar* → *validar* reproduzível e com **regressão zero verificável automaticamente**.
- **Trilha de commits (protocolo de rollback):**
 - `d1deb7c` — *CHECKPOINT 2026-08-12 16:44 — ANTES de implementar a FASE 0* (descreve a fase).
 - `e7b595f` — *FASE 0 CONCLUÍDA E VALIDADA* (com o resultado dos testes).
- **Fase 1 — a seguir:** implementar o **contrato de saída (JSON Schema) por task agêntica + validação/ reparo no ws-server** (Inserção A da v3). **Muda o runtime do gerador** — por isso terá seu próprio checkpoint *ANTES da Fase 1*.

2. Fase 0 — o que foi feito

Ferramental versionado em `tools/regen/` (runtime do gerador **inalterado**):

Arquivo	Função
<code>config.env</code>	Configuração (paths, <code>PROJECT_ID</code> , portas, endpoint do LLM, banco do app) — editável p/ outro app-teste
<code>regen_screens.py</code>	Regenera <code>frontend/src/screens/</code> do app-teste (determinístico, sem LLM)
<code>regen_adapters_tail.py</code>	Regenera a cauda auto-gerada do <code>ws-server/adapters.py</code> (helpers + adapters determinísticos + CRUD)
<code>regen.sh</code>	Orquestra a regeneração; limpa <code>__pycache__</code> antes (resolve o bug de import em cache)
<code>services.sh</code>	Sobe/derruba ws-server + frontend idempotente ; aponta o <code>LMSTUDIO_API_BASE</code> alcançável
<code>smoke_e2e.js</code>	E2E headless (Playwright): triagem→pré-diagnóstico→encaminhamento→prontuário→consulta; grava <code>e2e-carry.json</code>
<code>verify_chain.py</code>	Confere no banco que a cadeia persistiu ligada (FKs corretas)
<code>smoke.sh</code>	Smoke completo: serviços + E2E + verificação (sai 0 = verde)
<code>README.md</code>	Uso do harness

Problema que resolve (dor real das rodadas anteriores): o scratchpad da sessão era **volátil** (scripts sumiam entre chamadas), os serviços caíam e a regeneração às vezes usava **import em cache**. Agora tudo é **versionado e reproduzível**.

3. Fase 0 — testes de validação (saídas reais)

3.1 Teste A — regeneração determinística (`./regen.sh all`)

```
[regen_adapters_tail] adapters.py regenerado e válido (9 entidades)
[regen_screens] 30 arquivos escritos (project='ClinIA – Clínica Médica Inteligente', schema=5177
chars)
[regen] OK (all)
```

✓ Regenera **do zero** (sem cache): 30 telas + a cauda do `adapters.py` (9 entidades), sintaticamente válida (`ast.parse`).

3.2 Teste B — smoke E2E pela UI (`./smoke.sh`)

Um paciente ("Smoke 092074") percorreu o fluxo inteiro pela **UI real**; o *atendimento corrente* (localStorage) acumulou os IDs a cada etapa:

```
[1] ok=true {paciente_id, atendimento_id}
[2] ok=true {+ pre_diagnostico_id}
[3] ok=true {+ encaminhamento_id}
[4] ok=true {+ prontuario_id}
[5] ok=true
```

Carry final (cadeia completa herdada, sem redigitar):

```
{ "paciente_id":"c4433349-...", "atendimento_id":"c4adc80a-...", "pre_diagnostico_id":"f09f3f72-...",
"encaminhamento_id":"f599a041-...", "prontuario_id":"fa5ccc7d-..." }
```

3.3 Teste C — verificação no banco `clinia_ops` (`verify_chain.py`)

```
[verify] atendimento=True (Smoke 092074) | encaminhamento(medico+esp)=True | prontuario(liga
pre_diag+enc)=True
[smoke] VERDE
```

✓ A cadeia clínica **persistiu ligada**: o prontuário referencia o `pre_diagnostico_id` e o `encaminhamento_id` corretos; o encaminhamento tem médico e especialidade. **Regressão zero**.

3.4 Critérios da Fase 0 — todos atendidos

Critério	Resultado
<code>regen.sh all</code> reescreve os artefatos do zero, sem erro	30 arquivos + adapters válidos
<code>smoke.sh</code> verde (E2E + cadeia no banco)	VERDE
Scripts versionados (sobrevivem a reinício de sessão)	commit <code>e7b595f</code>

4. Telas — Fase 0 não altera telas (prova de exercício da UI)

A **Fase 0 não modifica nenhuma tela** (é ferramental; o gerador de telas ficou intacto — a regeneração é idempotente). As capturas abaixo são **evidência de que o smoke-test passou pela UI real** do app gerado.

4.1 App gerado — menu reorganizado (Atendimento × Cadastros), inalterado pela Fase 0:

ClinIA — Clínica Médica Inteligente

ATENDIMENTO

Recepção & Triagem

Pré-atendimento por Especialista

Geração de Pré-diagnóstico

Seleção de Médico

Registro/Prontuário

Consulta Médica

Procedimento Manual

Dashboard KPIs

Gestão de Pré-Diagnósticos

Gestão de Encaminhamentos

CADASTROS

Cadastro de Pacientes

Consentimentos

Gestão de Agendas

Gestão de Agentes

Gestão de Especialidades

Gestão de Médicos

Gestão de Pacientes

Atendimentos

Consentimentos

Encaminhamentos

Especialidades

Medicos

Pacientes

Cadastro de Pacientes

UC-001 - executado por agente de IA

Executar com IA

ATENDIMENTO CORRENTE

paciente_id: c4433349... atendimento_id: c4adc88a... pre_diagnostico_id: f09f3f72... encaminhamento_id: f599a041... prontuario_id: fa5ccc7d...

Encerrar / novo atendimento

Nome

CPF

Data de Nascimento

Contato

Convênio

Dispara o agente cadastrar_paciente

4.2 Registro/Prontuário com o *atendimento corrente* do smoke — a cadeia inteira herdada no banner (paciente → atendimento → pré-diagnóstico → encaminhamento → prontuário), com os dropdowns *Pré-diagnóstico* e *Encaminhamento* pré-preenchidos:

ClinIA — Clínica Médica Inteligente

ATENDIMENTO

Recepção & Triagem

Pré-atendimento por Especialista

Geração de Pré-diagnóstico

Seleção de Médico

Registro/Prontuário

Consulta Médica

Procedimento Manual

Dashboard KPIs

Gestão de Pré-Diagnósticos

Gestão de Encaminhamentos

CADASTROS

Cadastro de Pacientes

Consentimentos

Gestão de Agendas

Gestão de Agentes

Gestão de Especialidades

Gestão de Médicos

Gestão de Pacientes

Atendimentos

Consentimentos

Encaminhamentos

Especialidades

Médicos

Pacientes

Pre Diagnosticos

Prontuarios

Admin / Petri

Registro/Prontuário

UC-006 - executado por agente de IA

Executar com IA

ATENDIMENTO CORRENTE

paciente_id: c4433349...

atendimento_id: c4adc80a...

pre_diagnostico_id: f09f3f72...

encaminhamento_id: f599a041...

prontuario_id: fa5ccc7d...

Encerrar / novo atendimento

Paciente ID

Smoke 092074

Atendimento ID

c4adc80a-9686-11f1-8a81-cbea323b9023

Queixa inicial

Classificação de urgência

Área de destino

Hipóteses

Nível de confiança (%)

Exames sugeridos

Médico ID

Selecione...

Prioridade

Resumo para o médico

Pre diagnostico

f09f3f72-9686-11f1-8a81-cbea323b9023

Encaminhamento

f599a041-9686-11f1-8a81-cbea323b9023

Dispara o agente registrar_prontuario

5. Fase 1 — o que será feito (Inserção A: contrato de saída)

Objetivo. Cada task **agêntica** ganha um **contrato de saída (JSON Schema)** derivado do `expected_output` + das colunas `NOT NULL` /tipo da entidade persistida. A resposta do agente é **normalizada** → **coagida** → **validada** no ws-server **antes** de virar `task_completed`; falta de campo obrigatório ⇒ **erro explícito** (fail-loud).

Por que (dor que resolve). Hoje, em `_execute_task` do template do ws-server (≈ linha 2695), sem `output_func` manda-se `{raw: <texto do agente>}` cru — daí os remendos `parseAgentResult` (frontend) e `_cv` (adapter), e campos faltando geram falha `NOT NULL` silenciosa. A Fase 1 troca **três band-aids por um contrato na fonte**.

Onde entra (arquivos/funções). - `backend/agents/langnetagents.py` : - **nova**

`_derive_output_schema(task_name, task_cfg, entity_model)` — parseia `expected_output` + cruza com `_schema_model`; required **só** para colunas `NOT NULL` (evita sobre-especificar — lição do SWE-bench Verified); resolve enum↔float pelo tipo do banco; injeta `output_schema`: por task no `tasks.yaml`. - **nova**
`_coerce_to_schema(raw, schema)` no template do ws-server (`_template_websocket_server.py`), aplicada **entre as linhas ~2695 e ~2697** de `_execute_task`: desembulha `{raw}` /string; coage por tipo; valida `required`; suporta campo `refusal` / `fallback` (roteia p/ `fallback_manual`). Falta de obrigatório → 1 retry com o schema no prompt; persistindo → `error` explícito, **sem** persistir.

O que será verificado e validado ao final da Fase 1. 1. **Unit:** `_coerce_to_schema` — `{raw: "..."} → objeto`; "alta" → 0.9; `dict` → JSON string; ausência de `hipoteses` → `faltantes=["hipoteses"]`. 2. **E2E (pré-diagnóstico):** frontend recebe **objeto limpo** (sem `{raw}`); `nivel_confianca` **numérico**; `smoke.sh` continua **VERDE** (regressão

zero). 3. **Fault-injection:** forçar resposta incompleta → ws-server devolve **error explícito** e **nada persiste** (query no banco confirma). 4. **Não sobre-especificação:** resposta válida com campo **opcional** ausente **não** é rejeitada.

Benefício. Elimina **na fonte** a família de bugs de saída (`{raw}` , `enum↔float`, campo faltando→ `NOT NULL`). Aposenta `parseAgentResult / _cv` como remendos. Combate ao *specification gaming* ("respondeu algo ≠ respondeu o contrato").

Protocolo. Antes de iniciar, farei o commit+push **CHECKPOINT — ANTES da Fase 1**, descrevendo no texto exatamente o que a Fase 1 implementa (para rollback seguro).

6. Conclusão

A **Fase 0** entregou a base de reprodutibilidade e prova automatizada (regen + smoke), **validada 100%** e **sem alterar telas**. A partir dela, cada fase seguinte tem **regressão zero verificável** por um único comando (`./smoke.sh`). A **Fase 1** (contrato de saída) é a primeira a mexer no runtime do gerador e será precedida pelo seu checkpoint de rollback.